UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Innovación Basada en la Información y el Conocimiento



Ingeniería En Computación

PRÁCTICA 2

Planificación de hilos y orden de impresión

Hilos que imprimen del 1 al 10 y comparación con la ejecución secuencial

Nombre: GONZALEZ MARTINEZ ALONDRA GORETTI	**Código**: 218534681
Nombre de la clase: Programación paralela y recurrente	**Nombre del profesor:** Mtro. Zeus Cobián
Atributos de la competencia: Diseñar, implementar y probar algoritmos paralelos y concurrentes	**Actividad**: Práctica 2
Fecha de entrega: 17 de octubre del 2026	**Lenguaje**: Python 3.12

Bibliografía:

Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). *Operating System Concepts*. John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems*. Pearson Education.

Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation.

Recursos didácticos de la asignatura.

Contenido

Resumen . . . 3

1. Introducción . . . 3

Objetivos específicos . . . 4

2. Sustento teórico . . . 4

2.1 Planificación de hilos . . . 4

2.2 Qué garantiza y qué no garantiza un hilo . . . 4

2.3 La pausa con sleep y el GIL . . . 4

2.4 Concurrencia y salida compartida . . . 4

3. Diseño de la solución . . . 5

3.1 Qué hace el programa . . . 5

3.2 Funciones y opciones . . . 5

3.3 Pasos de la implementación . . . 6

4. Código fuente . . . 7

5. Capturas de resultados . . . 9

5.1 Ejecución secuencial . . . 9

5.2 Ejecución con 3 hilos . . . 9

5.3 Varias ejecuciones: el orden cambia . . . 10

5.4 Con pausas aleatorias . . . 12

5.5 Tiempos . . . 13

6. Reflexión: respuestas a las tres preguntas . . . 13

6.1 ¿Qué pasó con el orden de los números? . . . 13

6.2 ¿Por qué no siempre salen igual? . . . 14

6.3 ¿Qué ventaja tiene usar hilos en problemas más grandes? . . . 14

7. Conclusión . . . 15

Referencias . . . 16

Resumen

Este reporte documenta un programa en Python que crea 2 o 3 hilos, cada uno de los cuales imprime los números del 1 al 10 con una pequeña pausa (`sleep`) para simular tiempo de ejecución. El programa se ejecutó varias veces para observar cómo cambia el orden de impresión según la planificación de los hilos y se comparó con una ejecución secuencial. Se incluyen el sustento teórico, el código con resaltado de sintaxis, las capturas de los resultados y las respuestas a las tres preguntas de reflexión.

3.0 s → 1.0 s
tiempo con 3 tareas: secuencial vs. con 3 hilos

4 de 5
órdenes de impresión distintos en 5 repeticiones (3 hilos)

1 orden fijo
en la ejecución secuencial: A1 … A10, B1 … B10, C1 … C10

Tabla 0. Datos generales de la práctica.

Concepto	Detalle
Alumna	Gonzalez Martinez Alondra Goretti · código 218534681
Materia	Programación paralela y recurrente
Profesor	Mtro. Zeus Cobián
Actividad	Práctica 2 · Planificación de hilos y orden de impresión (100 puntos)
Fecha de entrega	17 de octubre del 2026
Archivos entregados	informe en PDF y `hilos_impresion.py` (programa)

Cómo leer este reporte
La sección 3 explica el diseño con diagramas; la 4 contiene el código; la 5 reúne las capturas de resultados; la 6 responde las tres preguntas de reflexión y la 7 resume las conclusiones.

1. Introducción

Cuando un programa tiene varios hilos, el orden exacto en que se ejecutan sus instrucciones no lo decide el programador, sino el **planificador** (*scheduler*) del sistema operativo, que reparte el tiempo del procesador entre los hilos listos para correr (Silberschatz et al., 2020). Por eso un mismo programa con hilos puede producir salidas distintas en ejecuciones distintas.

Esta práctica lo hace observable con un caso mínimo: cada hilo imprime los números del 1 al 10 con una pausa entre uno y otro. Al ejecutar el programa varias veces se ve cómo se mezclan las salidas de los hilos y se contrasta con la versión secuencial, donde el orden siempre es el mismo.

Objetivo general
Observar el comportamiento no determinista de la planificación de hilos y comparar el orden de impresión y el tiempo total frente a una ejecución secuencial.

Objetivos específicos

- Crear 2 o 3 hilos que impriman los números del 1 al 10 con una pausa que simule tiempo de ejecución.
- Ejecutar el programa varias veces y registrar cómo cambia el orden de impresión.
- Comparar con una ejecución secuencial en orden de salida y en tiempo total.
- Responder: qué pasó con el orden, por qué no siempre sale igual y qué ventaja dan los hilos en problemas más grandes.

2. Sustento teórico

2.1 Planificación de hilos

En un sistema operativo moderno hay más hilos listos para ejecutarse que núcleos disponibles, así que el planificador alterna cuál corre y durante cuánto tiempo. Sus decisiones dependen de la carga del sistema, de las prioridades y de cuándo despierta cada hilo, y el programa no las controla. Por eso el orden de ejecución entre hilos es **no determinista**: puede cambiar de una ejecución a otra aunque el código y los datos sean idénticos (Tanenbaum y Bos, 2015).

2.2 Qué garantiza y qué no garantiza un hilo

Tabla 1. Lo que está garantizado y lo que no en un programa con hilos.

Aspecto	¿Garantizado?	Explicación
Orden de las instrucciones dentro de un hilo	**Sí**	Cada hilo ejecuta su código en secuencia: imprime 1, luego 2, luego 3…
Orden entre instrucciones de hilos distintos	**No**	Lo decide el planificador; puede cambiar en cada ejecución.
Que un hilo termine antes que otro	**No**	Depende de cuándo se le asigna tiempo de procesador.
Que el programa espere a todos los hilos	**Sí, con join()**	El hilo principal se bloquea hasta que el hilo termina.

2.3 La pausa con sleep y el GIL

`time.sleep()` suspende únicamente al hilo que lo llama durante el tiempo indicado (Python Software Foundation, 2024). Mientras un hilo duerme, CPython libera el *Global Interpreter Lock* (GIL) y otro hilo puede ejecutarse. Por eso, a diferencia de la suma de la Práctica 1 (trabajo de cálculo puro), aquí los hilos sí pueden **traslapar sus esperas** y reducir el tiempo total: la pausa simula una operación de entrada/salida, donde los hilos son útiles.

2.4 Concurrencia y salida compartida

Los hilos comparten la salida estándar. Si un hilo escribe una línea en dos pasos (por ejemplo, el texto y luego el salto de línea), otro hilo puede intercalarse en medio y mezclar caracteres. Para evitarlo, el programa escribe cada línea completa en una sola llamada (`sys.stdout.write`).



Figura 1. Comparación conceptual: tres tareas en secuencia frente a tres hilos que avanzan a la vez. *Fuente: elaboración propia.*

3. Diseño de la solución

3.1 Qué hace el programa

El programa ejecuta primero la versión secuencial (las tareas A, B y C una tras otra) y después la versión con hilos, repetida las veces que se indique. En cada repetición registra el orden en que se imprimió cada línea y comprueba que cada hilo haya conservado su propio orden del 1 al 10.

3.2 Funciones y opciones

Tabla 2. Funciones del programa `hilos_impresion.py`.

Función	Responsabilidad
`tarea(nombre, pausa, aleatoria, registro)`	Imprime del 1 al 10 con una pausa; anota cada impresión en una lista compartida.
`ejecutar_secuencial(...)`	Ejecuta las tareas una detrás de otra, sin hilos, y mide el tiempo.
`ejecutar_con_hilos(...)`	Crea un hilo por tarea, los inicia, espera con join() y mide el tiempo.
`leer_argumentos()`	Lee las opciones de la línea de comandos.
`main()`	Orquesta las ejecuciones, verifica el orden interno y muestra el resumen.

Tabla 3. Opciones de la línea de comandos.

Opción	Efecto	Valor por defecto
`-n / --hilos`	Número de hilos (2 o 3).	3
`-r / --repeticiones`	Veces que se repite la versión con hilos.	1
`-p / --pausa`	Pausa en segundos entre números.	0.1
`-a / --aleatoria`	Pausa aleatoria entre 0.5 y 1.5 veces la pausa base.	desactivada

3.3 Pasos de la implementación

1

Definir la tarea

Cada hilo ejecuta la misma función: imprime del 1 al 10 y espera un poco antes de cada impresión para simular tiempo de trabajo.

```
26  def tarea(nombre, pausa, aleatoria, registro):
27      """Trabajo de cada hilo: imprime del 1 al 10 con una pausa entre números."""
28      for numero in range(1, LIMITE + 1):
29          time.sleep(random.uniform(0.5 * pausa, 1.5 * pausa) if aleatoria else pausa)
30          # Se escribe la línea completa en una sola llamada para que no se
31          # mezclen los caracteres de dos hilos en la misma línea.
32          sys.stdout.write(f"  Hilo {nombre}: {numero}\n")
```

2

Crear los hilos

Se crea un objeto Thread por cada tarea (A, B y C). Todavía no se ejecutan.

```
48      hilos = [threading.Thread(target=tarea, args=(nombre, pausa, aleatoria, registro))
49               for nombre in NOMBRES[:n_hilos]]
```

3

Iniciar los hilos

Con start() cada hilo comienza a ejecutarse; desde ese momento el planificador decide cuál avanza en cada instante.

```
51      for hilo in hilos:
52          hilo.start()
```

4

Esperar con join()

El hilo principal espera a que termine cada hilo antes de continuar y de medir el tiempo total.

```
53      for hilo in hilos:
54          hilo.join()
```

5

Repetir y comparar

La versión con hilos se repite r veces; se guarda cada orden de impresión y al final se cuenta cuántos órdenes distintos aparecieron.

```
83          ordenes.append(" ".join(orden))
84          tiempos.append(t)
85          # Cada hilo siempre conserva su propio orden 1..10; lo que cambia es la mezcla entre hilos
```

```
96      print(f"\nÓrdenes distintos entre repeticiones: {len(set(ordenes))} de {len(ordenes)}")
```

4. Código fuente

Listado completo de `hilos_impresion.py`. Para ejecutarlo:

terminal · cómo ejecutar

```
$ python hilos_impresion.py              # 3 hilos, 1 repetición
$ python hilos_impresion.py -n 2 -r 5       # 2 hilos, 5 repeticiones
$ python hilos_impresion.py -n 3 -r 5 -a    # pausas aleatorias (más variación)
```

hilos_impresion.py · Python 3

```
"""Práctica 2: Planificación de hilos y orden de impresión.
Alumna: Alondra Goretti González Martínez (218534681)
Materia: Programación paralela y recurrente - Mtro. Zeus Cobián

Cada hilo imprime los números del 1 al 10 con una pequeña pausa (sleep) que
simula tiempo de ejecución. El programa compara:
  1) una ejecución secuencial (sin hilos), y
  2) una ejecución con 2 o 3 hilos, repetida varias veces para observar
     cómo cambia el orden de impresión según la planificación.

Uso:
    python hilos_impresion.py                   # 3 hilos, 1 repetición
    python hilos_impresion.py -n 2 -r 5         # 2 hilos, 5 repeticiones
    python hilos_impresion.py -n 3 -r 5 -a      # pausas aleatorias (más variación)
"""
import argparse
import random
import sys
import threading
import time

NOMBRES = ["A", "B", "C"]
LIMITE = 10


def tarea(nombre, pausa, aleatoria, registro):
    """Trabajo de cada hilo: imprime del 1 al 10 con una pausa entre números."""
    for numero in range(1, LIMITE + 1):
        time.sleep(random.uniform(0.5 * pausa, 1.5 * pausa) if aleatoria else pausa)
        # Se escribe la línea completa en una sola llamada para que no se
        # mezclen los caracteres de dos hilos en la misma línea.
        sys.stdout.write(f"  Hilo {nombre}: {numero}\n")
        registro.append(f"{nombre}{numero}")  # list.append es seguro entre hilos en CPython


def ejecutar_secuencial(n_hilos, pausa, aleatoria):
    """Las mismas tareas, una detrás de otra, sin hilos."""
```

hilos_impresion.py · Python 3

```python
    registro = []
    t0 = time.perf_counter()
    for nombre in NOMBRES[:n_hilos]:
        tarea(nombre, pausa, aleatoria, registro)
    return registro, time.perf_counter() - t0


def ejecutar_con_hilos(n_hilos, pausa, aleatoria):
    """Las mismas tareas, cada una en su propio hilo."""
    registro = []
    hilos = [threading.Thread(target=tarea, args=(nombre, pausa, aleatoria, registro))
             for nombre in NOMBRES[:n_hilos]]
    t0 = time.perf_counter()
    for hilo in hilos:
        hilo.start()
    for hilo in hilos:
        hilo.join()
    return registro, time.perf_counter() - t0


def leer_argumentos():
    p = argparse.ArgumentParser(description="Hilos que imprimen del 1 al 10")
    p.add_argument("-n", "--hilos", type=int, default=3, choices=[2, 3], help="número de hilos (2 o 3)")
    p.add_argument("-r", "--repeticiones", type=int, default=1, help="veces que se repite la versión con hilos")
    p.add_argument("-p", "--pausa", type=float, default=0.1, help="pausa en segundos entre números (default 0.1)")
    p.add_argument("-a", "--aleatoria", action="store_true", help="pausa aleatoria entre 0.5x y 1.5x de --pausa")
    return p.parse_args()


def main():
    args = leer_argumentos()
    n, pausa, alea = args.hilos, args.pausa, args.aleatoria

    print("=" * 56)
    print(f"EJECUCIÓN SECUENCIAL (sin hilos) · {n} tareas")
    print("=" * 56)
    seq, t_seq = ejecutar_secuencial(n, pausa, alea)
    print(f"Tiempo secuencial: {t_seq:.2f} s\n")

    ordenes, tiempos = [], []
    for rep in range(1, args.repeticiones + 1):
        print("=" * 56)
        print(f"EJECUCIÓN CON {n} HILOS · repetición {rep} de {args.repeticiones}")
        print("=" * 56)
        orden, t = ejecutar_con_hilos(n, pausa, alea)
        ordenes.append(" ".join(orden))
        tiempos.append(t)
        # Cada hilo siempre conserva su propio orden 1..10; lo que cambia es la mezcla entre hilos
        ok = all([x for x in orden if x[0] == nombre] == [f"{nombre}{i}" for i in range(1, LIMITE + 1)]
                 for nombre in NOMBRES[:n])
        print(f"Tiempo con hilos: {t:.2f} s · orden interno de cada hilo correcto: {'sí' if ok else 'NO'}\n")

    print("=" * 56)
    print("RESUMEN")
    print("=" * 56)
    print(f"Orden secuencial : {' '.join(seq)}")
    for i, o in enumerate(ordenes, 1):
        print(f"Con hilos (rep {i}): {o}")
    print(f"\nÓrdenes distintos entre repeticiones: {len(set(ordenes))} de {len(ordenes)}")
    print(f"Tiempo secuencial: {t_seq:.2f} s | tiempo medio con hilos: {sum(tiempos) / len(tiempos):.2f} s")


if __name__ == "__main__":
    main()
```

5. Capturas de resultados

5.1 Ejecución secuencial

Sin hilos, el orden es siempre el mismo: primero termina la tarea A, luego la B y luego la C. El tiempo es la suma de las tres (≈ 3 s).



Figura 2. Captura de la parte secuencial de `python hilos_impresion.py -n 3`. *Fuente: elaboración propia.*

5.2 Ejecución con 3 hilos

Con hilos, las tres tareas avanzan casi a la vez y sus líneas se intercalan. Cada hilo conserva su propio orden (1, 2, 3, …), pero la mezcla entre hilos es distinta. El programa verifica ese orden interno y reporta el tiempo total (≈ 1 s).

```
terminal — hilos_impresion.py

$ python hilos_impresion.py -n 3   (parte con hilos)
============================================================
EJECUCIÓN CON 3 HILOS · repetición 1 de 1
============================================================
  Hilo A: 1
  Hilo B: 1
  Hilo C: 1
  Hilo C: 2
  Hilo B: 2
  Hilo A: 2
  Hilo B: 3
  Hilo A: 3
  Hilo C: 3
  Hilo A: 4
  Hilo C: 4
  Hilo B: 4
  Hilo C: 5
  Hilo A: 5
  Hilo B: 5
  Hilo A: 6
  Hilo C: 6
  Hilo B: 6
  Hilo C: 7
  Hilo A: 7
  Hilo B: 7
  Hilo A: 8
  Hilo B: 8
  Hilo C: 8
  Hilo C: 9
  Hilo B: 9
  Hilo A: 9
  Hilo B: 10
  Hilo A: 10
  Hilo C: 10
Tiempo con hilos: 1.00 s · orden interno de cada hilo correcto: sí
```

Figura 3. Captura de la parte con hilos (una repetición) del mismo comando. *Fuente: elaboración propia.*

5.3 Varias ejecuciones: el orden cambia

Se repitió la versión con hilos varias veces. Estos son los resúmenes que imprime el programa; cada línea es el orden completo de impresión de una repetición.

```
terminal · 3 hilos, 5 repeticiones

$ python hilos_impresion.py -n 3 -r 5
Orden secuencial : A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 B1 B2 B3 B4 B5 B6 B7 B8 B9 B10 C1 C2 C3 C4 C5 C6 C7 C8 C9 C10
Con hilos (rep 1): A1 B1 C1 C2 B2 A2 B3 A3 C3 A4 C4 B4 C5 B5 A5 B6 A6 C6 A7 C7 B7 B8 C8 A8 C9 A9 B9 A10 B10 C10
Con hilos (rep 2): B1 C1 A1 A2 C2 B2 C3 B3 A3 B4 A4 C4 A5 C5 B5 C6 B6 A6 B7 A7 C7 A8 C8 B8 C9 B9 A9 B10 A10 C10
Con hilos (rep 3): B1 C1 A1 A2 C2 B2 C3 B3 A3 B4 A4 C4 A5 C5 B5 C6 B6 A6 B7 A7 C7 A8 C8 B8 C9 B9 A9 B10 A10 C10
Con hilos (rep 4): C1 A1 B1 B2 A2 C2 A3 C3 B3 C4 B4 A4 A5 B5 C5 B6 C6 A6 C7 A7 B7 B8 A8 C8 A9 C9 B9 C10 B10 A10
Con hilos (rep 5): C1 A1 B1 B2 A2 C2 A3 C3 B3 C4 B4 A4 B5 A5 C5 A6 C6 B6 C7 B7 A7 B8 A8 C8 A9 C9 B9 C10 B10 A10

Órdenes distintos entre repeticiones: 4 de 5
Tiempo secuencial: 3.00 s | tiempo medio con hilos: 1.00 s
```

Figura 4. Resumen de 5 repeticiones con 3 hilos. *Fuente: elaboración propia.*

```
terminal · 2 hilos, 4 repeticiones

$ python hilos_impresion.py -n 2 -r 4
Orden secuencial : A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 B1 B2 B3 B4 B5 B6 B7 B8 B9 B10
Con hilos (rep 1): A1 B1 B2 A2 B3 A3 A4 B4 B5 A5 A6 B6 A7 B7 B8 A8 B9 A9 A10 B10
Con hilos (rep 2): A1 B1 B2 A2 B3 A3 A4 B4 B5 A5 A6 B6 B7 A7 A8 B8 B9 A9 A10 B10
Con hilos (rep 3): B1 A1 A2 B2 A3 B3 B4 A4 A5 B5 B6 A6 A7 B7 B8 A8 A9 B9 B10 A10
Con hilos (rep 4): A1 B1 B2 A2 B3 A3 A4 B4 B5 A5 A6 B6 B7 A7 A8 B8 B9 A9 A10 B10
Órdenes distintos entre repeticiones: 3 de 4
Tiempo secuencial: 2.00 s | tiempo medio con hilos: 1.00 s
```

Figura 5. Resumen de 4 repeticiones con 2 hilos. *Fuente: elaboración propia.*



Figura 6. La fila superior es la ejecución secuencial; las demás son repeticiones con hilos. Cada color es un hilo y cada casilla una línea impresa. *Fuente: elaboración propia.*

5.4 Con pausas aleatorias

Con la opción `-a` cada pausa dura entre 0.5 y 1.5 veces la base. Los hilos ya no despiertan casi al mismo tiempo y la mezcla varía más.



Figura 7. Seis repeticiones con pausas aleatorias. *Fuente: elaboración propia.*

Tabla 4. Órdenes distintos observados y tiempos medios.

Experimento	Repeticiones	Órdenes distintos	Tiempo medio con hilos	Tiempo secuencial
3 hilos, pausa fija (captura 5.3)	5	4	1.00 s	3.00 s
2 hilos, pausa fija (captura 5.3)	4	3	1.00 s	2.00 s
3 hilos, pausa fija (figura)	8	6	≈ 1.00 s	≈ 3.00 s
3 hilos, pausa aleatoria (figura)	8	8	≈ 1.1 s	≈ 3.0 s
3 hilos, pausa aleatoria (captura)	4	4	1.09 s	2.82 s

Nota: con pausa fija algunas repeticiones coinciden por casualidad (por eso 4 de 5 o 3 de 4); con pausa aleatoria casi todas difieren.

5.5 Tiempos

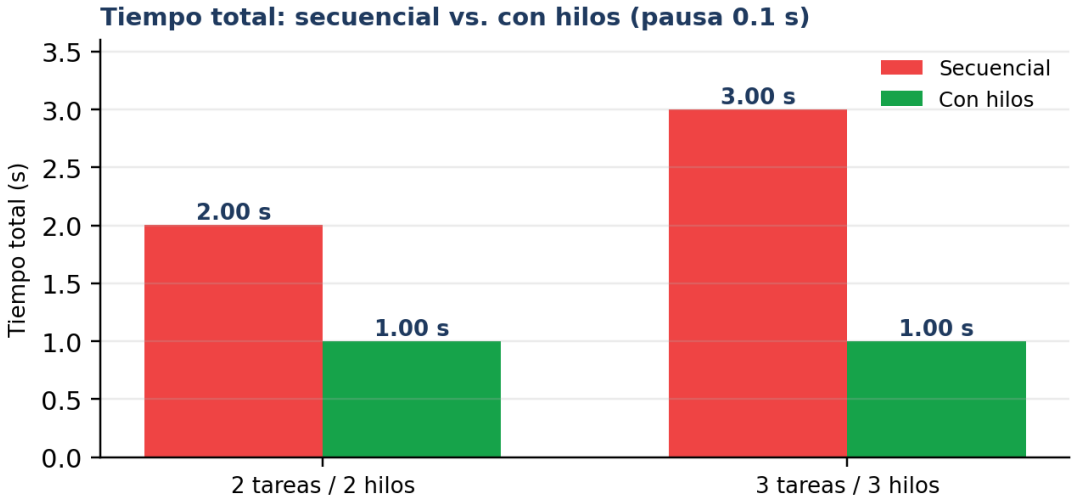


Figura 8. Tiempo total de la versión secuencial y de la versión con hilos. *Fuente: elaboración propia.*

6. Reflexión: respuestas a las tres preguntas

6.1 ¿Qué pasó con el orden de los números?

Hay dos niveles. **Dentro de cada hilo** el orden siempre fue correcto: A imprimió 1, 2, 3, …, 10, y lo mismo B y C (el programa lo verifica en cada repetición). **Entre hilos**, en cambio, las líneas se mezclaron: en lugar de A1…A10, B1…B10, C1…C10 como en la versión secuencial, aparecieron intercaladas, por ejemplo `A1 B1 C1 C2 B2 A2 ….` Además, la mezcla no fue la misma en todas las ejecuciones: en cada «ronda» (el número k de cada hilo) cambió cuál hilo salió primero.

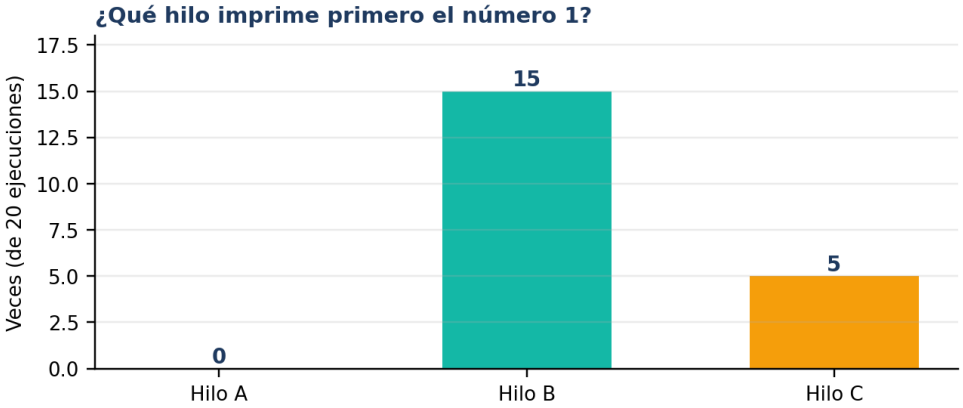


Figura 9. Hilo que imprimió primero el número 1 en 20 ejecuciones con 3 hilos y pausa fija. *Fuente: elaboración propia.*

Resultado de la observación

En 20 ejecuciones, el número 1 lo imprimió primero el hilo B en 15 ocasiones y el C en 5; el A no salió primero en este lote, aunque sí lo hizo en otras corridas (por ejemplo, en la captura 5.2). Es decir, hay tendencias, pero ninguna regla fija: el orden no se puede predecir.

6.2 ¿Por qué no siempre salen igual?

Porque el orden entre hilos lo decide el **planificador del sistema operativo**, no el programa. Los tres hilos duermen la misma pausa y despiertan casi al mismo tiempo; la diferencia de unos microsegundos (en qué momento se creó cada hilo con `start()`, la carga del sistema en ese instante, cuál obtuvo primero el turno) basta para que uno imprima antes que otro. Como no hay ninguna sincronización que obligue un orden entre hilos, cada ejecución puede resultar distinta. Con pausas aleatorias el efecto se amplifica, porque los hilos ya no despiertan juntos.

Esta variabilidad no es un error del programa: es una propiedad de la programación con hilos. Si el orden importara, habría que imponerlo con mecanismos de sincronización (`join()`, candados o colas), a costa de reducir la concurrencia.

6.3 ¿Qué ventaja tiene usar hilos en problemas más grandes?

La ventaja principal es que el tiempo total deja de ser la *suma* de las tareas y pasa a parecerse al de la *más larga*, porque los hilos traslapan sus esperas. Con 3 tareas de 1 s cada una, la versión secuencial tardó ≈ 3 s y la de hilos ≈ 1 s. Al crecer el problema, la diferencia aumenta: con 8 tareas, el tiempo secuencial subió a 4.0 s mientras el de hilos se mantuvo en 0.5 s.

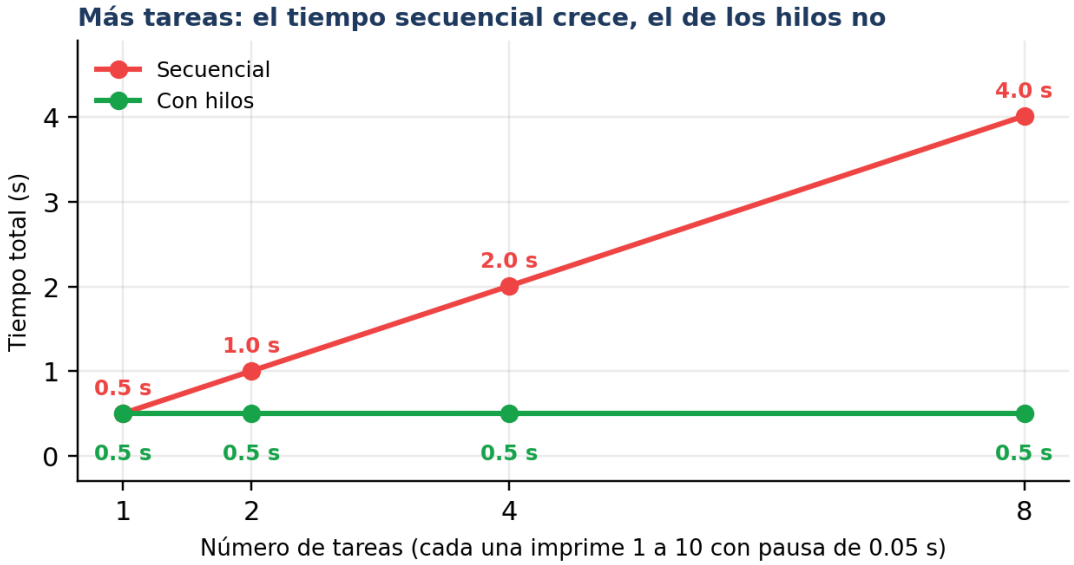


Figura 10. Tiempo total al aumentar el número de tareas (cada una imprime 1 a 10 con pausa de 0.05 s). *Fuente: elaboración propia.*

Tabla 5. Tiempo medido al aumentar el número de tareas.

Tareas	Secuencial (s)	Con hilos (s)	Aceleración
1	0.50	0.50	1.0×
2	1.00	0.50	2.0×
4	2.01	0.50	4.0×
8	4.01	0.50	8.0×

Una advertencia importante

Esta ventaja aparece porque la pausa simula **entrada/salida** (esperar red, disco o usuario), donde CPython libera el GIL. En trabajo de cálculo puro, como la suma de la Práctica 1, los hilos de Python no aceleran: allí los tiempos con 1 a 32 hilos fueron prácticamente iguales (≈ 23 ms). Para acelerar cálculo intensivo conviene usar procesos (`multiprocessing`).

Tabla 6. Cuándo conviene usar hilos en Python.

Tipo de problema	Ejemplo	¿Ayudan los hilos en CPython?
Espera de entrada/salida	Descargas, lectura de archivos, consultas a bases de datos, peticiones de red	**Sí**: las esperas se traslapan
Cálculo intensivo (CPU)	Sumar millones de números, procesar imágenes	**No**: el GIL lo impide; usar procesos
Aplicaciones interactivas	Interfaces que no deben congelarse mientras trabajan	**Sí**: la interfaz sigue respondiendo

También hay costos: el orden se vuelve impredecible, depurar es más difícil y, si los hilos comparten datos, hacen falta mecanismos de sincronización para evitar condiciones de carrera.

7. Conclusión

Orden

Cada hilo conserva su orden interno; entre hilos las líneas se intercalan y la mezcla cambia entre ejecuciones.

Causa

El planificador del sistema operativo decide qué hilo corre y cuándo; el programa no controla ese orden.

Tiempo

Con pausas, 3 hilos terminaron en ≈ 1 s frente a ≈ 3 s en secuencial: los hilos traslapan las esperas.

Límite

En cálculo puro, el GIL de CPython evita esa ganancia (Práctica 1); ahí conviene usar procesos.

La práctica mostró de forma directa que la ejecución con hilos es *no determinista*: el mismo programa, con los mismos datos, imprimió las líneas en órdenes distintos en distintas ejecuciones, mientras que la versión secuencial siempre produjo el mismo resultado. Lo único que se mantuvo fijo fue el orden interno de cada hilo.

También se comprobó que, cuando el trabajo consiste en esperas, usar hilos reduce el tiempo total de la suma de las tareas al de la más larga, y que la ventaja crece con el tamaño del problema. Sin embargo, esa ventaja depende del tipo de trabajo: en cálculo intensivo, CPython no la ofrece por el GIL. En mi opinión, lo más importante que deja esta práctica es entender que, al usar hilos, hay que diseñar los programas para que su correctitud no dependa del orden de ejecución, y usar sincronización solo donde ese orden sí importe.

Referencias

Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation. https://docs.python.org/3/library/threading.html

Python Software Foundation. (2024). *time — Time access and conversions*. Python 3 Documentation. https://docs.python.org/3/library/time.html

Silberschatz, A., Galvin, P. B. y Gagne, G. (2020). *Operating System Concepts* (10.ª ed.). John Wiley & Sons.

Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson Education.